



Security Audit

Noon Capital 4th (Governance Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	60
Conclusion	61
Our Methodology	62
Disclaimers	64
About Hashlock	65

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Noon Capital team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Noon Capital is a DeFi governance protocol built on the Ethereum Network that allows users to stake NOON tokens and participate in decentralized governance through a veToken system. The protocol features time-locked staking with voting power multipliers, VIP staking tiers with enhanced rewards, and a comprehensive vesting mechanism for long-term token distribution.

In simple terms, users can stake NOON tokens to gain voting power in governance decisions while earning additional rewards through the vesting system, creating aligned incentives for long-term participation in the protocol.

Project Name: Noon Capital

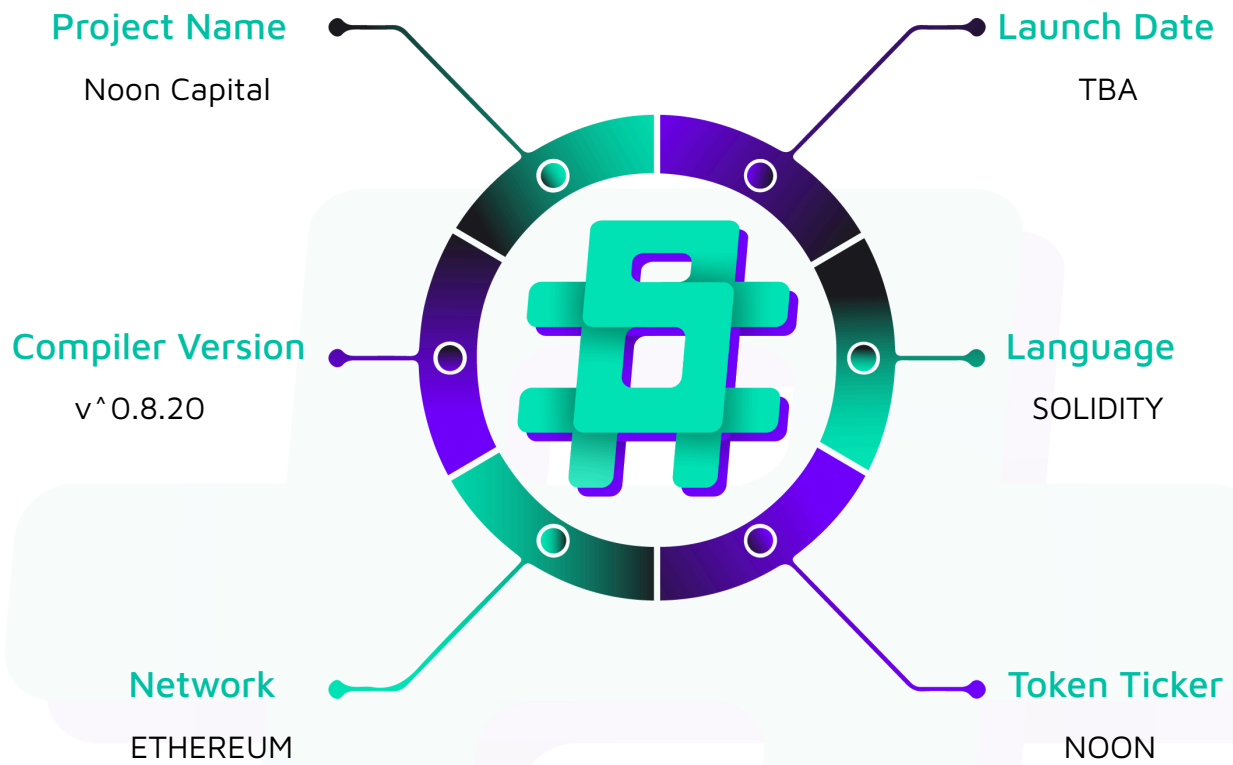
Project Type: Governance Token

Compiler Version: ^0.8.20

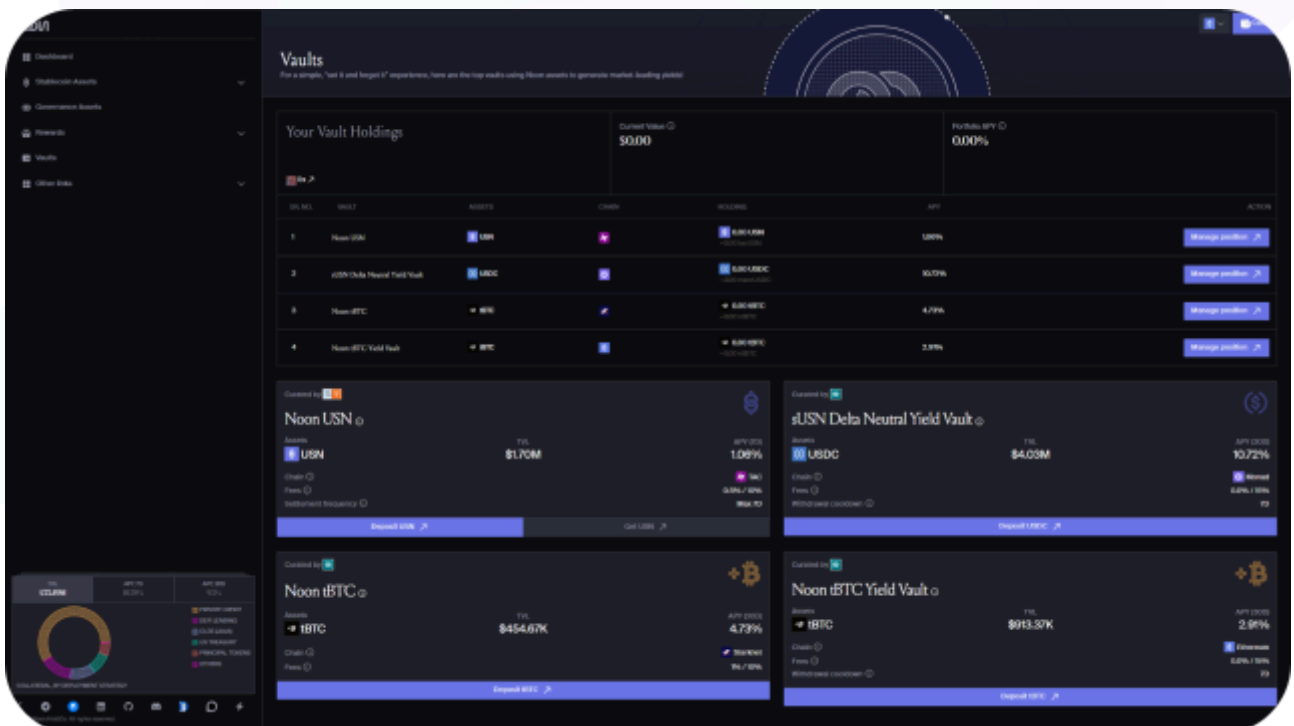
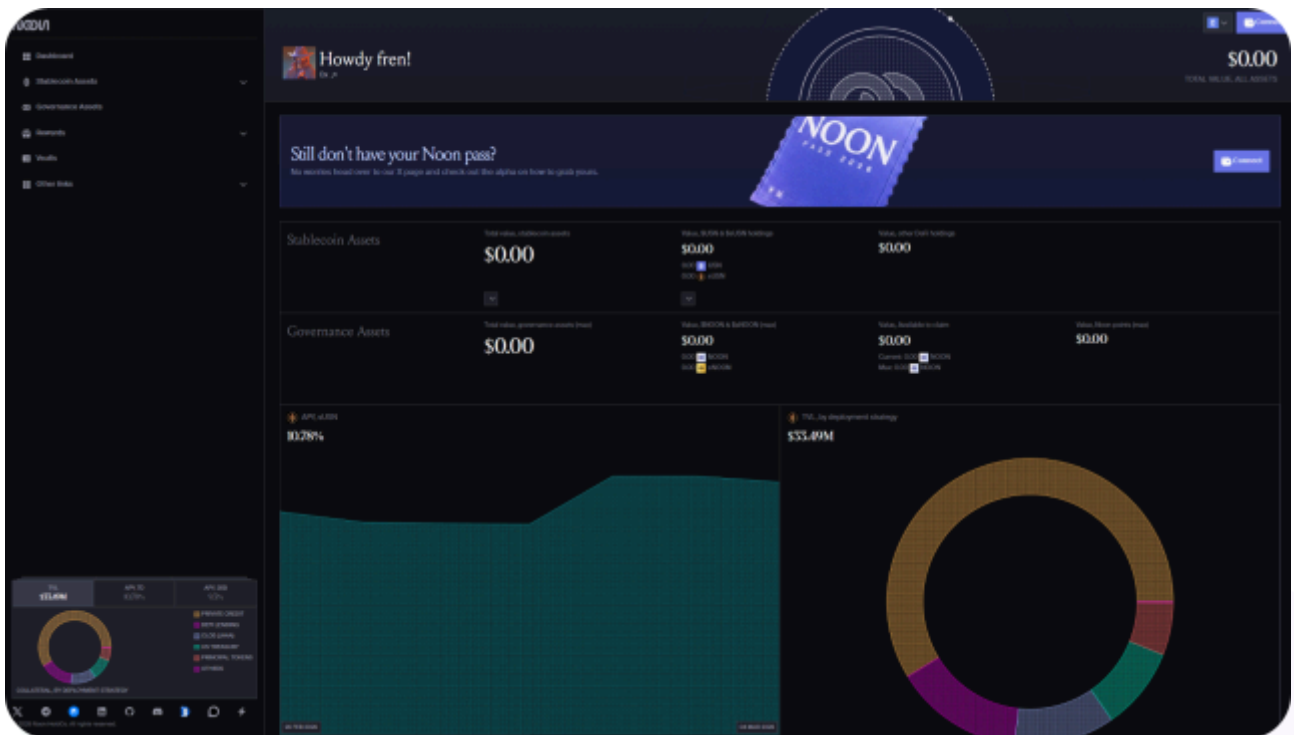
Website: <https://noon.capital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Noon Capital project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Noon Capital Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	September, 2026
Contract 1	Bridge rate limiter v1
Audited GitHub Commit Hash	df2225c4342709f0040d85d6b2366b1424c278af
Fix Review GitHub Commit Hash	81ef548cb05a6c24e8e9bc8dc70616befe1b6b05

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

4 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Bridge rate limiter v1 - Adds one shared rate-limiter contract on each chain. USN, sUSN, StakedUSNHyperlane and the staking vault all use it for their bridge transfers. - Limits both LayerZero and Hyperlane transfers, in both directions (sending and receiving). - Checks the limit before tokens are burned or locked when sending, and before they are minted or unlocked when receiving. - Keeps separate limits per contract, per bridge, per remote chain and per direction. - Limits refill steadily over time (sliding window), the same way LayerZero's rate limiter works. - A time window of 0 turns the limit into a per-transaction cap. - Tokens without a limiter set keep bridging as they do today. - Allows the owner to block new outgoing transfers for a contract while incoming transfers keep arriving. - Allows the owner to set limits, reset usage and change ownership in two steps. - Changing a limit keeps the usage built up under the old settings. - Supports very large limits. - Adds the limiter address to each upgradeable token while keeping its existing storage unchanged.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Noon Capital project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Noon Capital project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] StakingVaultOFTUpgradeableHyperlane#validateExecuteHandleIssue

- Stuck-message reconciliation does not consume the original bridge message

Description

The vault has a manual escape hatch for cross-chain messages that cannot be delivered: an admin records an arbitrary (guid, amount) pair, waits 48 hours, and the vault then releases that many shares from its locked escrow pool to owner(). The release only updates the vault's own bookkeeping. It does not cancel or consume the LayerZero or Hyperlane message it is compensating for, so that message stays valid and can still be executed later. If it is, the same transfer is paid out twice from a shared pool.

Vulnerability Details

requestHandleFixIssue(guid, amount, reason) stores an admin-supplied guid and amount, and after the timelock validateExecuteHandleIssue moves that amount out of the locked pool:

```
uint256 amount = req.amount;

address recipient = owner();

if (balanceOf(address(this)) < amount) revert InsufficientLockedBalance();

...

req.executed = true;

_update(address(this), recipient, amount);

emit StuckMessageReconciled(guid, recipient, amount);
```

req.executed only prevents that same request from running twice. Nothing here touches the bridge. On LayerZero the packet's inboundPayloadHash is still stored in the

endpoint, because a reverted `IzReceive` rolls back the clearing. On Hyperlane the delivery record is written before `handle` is called, but a revert inside `handle` reverts `Mailbox.process` in full and rolls that write back, so `delivered(id)` stays false. Both retry paths are permissionless. `handle` and `_credit` never read `stuckMessageRequests`, so neither knows a reconciliation happened, and the `guid` is unvalidated and unrelated to any real packet identifier, so the same transfer can even be reconciled twice under two different labels.

A message only needs to become deliverable again for the second payout to occur — which happens routinely when a limit is raised, a peer is restored, or a paused token is unpaused. The retry can come from a relayer on its normal backoff, from any third party, or from the original user through LayerZero Scan after already being made whole off-chain.

The pause and blacklist case makes this the default outcome rather than a remote one. The payout runs through the same `_update` gate as normal delivery, so while the vault is paused or `address(this)` is blacklisted, `validateExecuteHandleIssue` itself reverts. The admin must clear that block to pay out, and clearing it is exactly what makes the stuck message deliverable again.

That leaves the separate cancellation as the whole of the protection. The contract never consumes the message; safety depends on an operator calling `endpoint.clear(...)` or `skip()` on the relevant nonce. In both production uses of this path to date that was done — the gap is that nothing in the contract enforces it.

Two further points: this code is live on Ethereum mainnet, not only on this branch, and reconciliation bypasses the rate limiter entirely (`handle` calls `_checkAndUpdateRateLimit` at L367, `validateExecuteHandleIssue` does not call it at all), so escrow released this way sits outside the inbound rate-limit accounting.

Impact

A reconciled transfer can be paid out a second time, leaving the vault's locked pool short by that amount. The loss is bounded by the size of the single reconciled message rather than the pool, and no attacker is involved. The reconciled user is unaffected, having already been made whole; the shortfall surfaces later, when another user bridges in and their delivery reverts with an insufficient balance error that does not

point at the cause. The pool can be topped up once identified, so the practical cost is the shortfall plus the time spent diagnosing an unrelated-looking failure.

Recommendation

Make the payout and the cancellation atomic. For LayerZero, call `endpoint.clear(...)` for the packet in the same transaction as the release; the vault's owner is already the endpoint delegate, so no additional authority is required. Derive the amount and the recipient from the packet rather than accepting them as parameters, and pay the decoded recipient instead of `owner()`.

Hyperlane needs a different approach. `handle` is not given the nonce — it exists on the wire but is not forwarded — so the contract cannot reconstruct the canonical message ID, and any key derived from what it does receive would collide between two identical transfers. Either carry a sequence number in the message body so a reconciled message can be identified on arrival, or do not reconcile Hyperlane messages without off-chain evidence that delivery is permanently impossible.

Operationally, do not use this path for stalls caused by a pause or a blacklist, since executing the payout requires clearing the condition that is holding the message back.

The team confirmed that executions of this path follow an off-chain process with multiple verifications, and that the related information is published publicly on their forum. On-chain records support this: both prior uses were paired with a cancellation of the underlying packet. The finding will be acknowledged on that basis.

Status

Acknowledged

Low

[L-01] `periphery/USNHyperlane` & `periphery/USNOFTHyperlane` - Rate limiter not applied to all bridge-enabled contracts

Description

The pull request describes the rate limiter as shared by every bridge-enabled contract on a chain, and wires it into four of them. Two further contracts mint on inbound bridge messages and were left out. Between them they are deployed three times across TAC and zkSync, and all three are live, so the control is applied inconsistently rather than protocol-wide as documented.

Vulnerability Details

Six contracts in `contracts/` expose a bridge path that changes supply. `USNUpgradeableHyperlane`, `StakingVaultOFTUpgradeableHyperlane`, `remote/StakedUSNHyperlane` and `remote/StakedUSNOFTHyperlane` each gained a `rateLimiter` address and a `_checkAndUpdateRateLimit` call in front of every burn, lock, mint and unlock. `periphery/USNHyperlane.sol` and `periphery/USNOFTHyperlane.sol` were left unchanged and still mint directly inside `handle` with no cap:

```
if (recipient == address(0)) revert InvalidRecipient();  
  
_mint(recipient, amount);
```

Three live deployments run this uncovered code: USN on TAC (0x51a30e64..., 3,882.06 USN in circulation), USN on zkSync Era (0x0469d9d1..., 30,522.63 USN), and sUSN on TAC (0x5CED7f73..., 536.40 sUSN). All three have `hyperlaneEnabled = true` and `rateLimiter()` reverts.

The third one is easy to miss. The sUSN route to zkSync terminates in `StakedUSNOFTHyperlane`, which this PR covers, but the sUSN route to TAC terminates in a third deployment of `periphery/USNHyperlane` (verified implementation 0x6d1019bd...). That address is the mainnet vault's registered `remoteTokens(239)`, so it sits on a live wired route while running uncovered code.

Neither periphery contract inherits Pausable, so the pause lever available on the four updated contracts does not exist there. `configureHyperlane(address(0))` and, on the OFT variant, `setPeer(eid, bytes32(0))` remain, but neither is clean: the first does not clear `hyperlaneEnabled`, it only breaks the route because `address(0)` can never be `msg.sender`; the second disables the LayerZero leg only.

Wiring them needs no migration. `rateLimiter` does not even take a new slot — it packs into free bytes in slot 4 beside the existing `hyperlaneEnabled` bool.

Impact

This is a consistency and completeness gap rather than an exploitable weakness. Inbound minting on TAC and `zkSync` stays uncapped, but the exposure is small in both directions: anything minted there has to travel back through mainnet USN to be realised, and that leg will be capped once this PR is deployed and limits are configured. Local exposure is bounded by liquidity on those chains — 34,404.69 USN in total, around 0.09% of mainnet supply, plus 536.40 sUSN.

The practical risk is operational. The documentation says the limiter covers every bridge-enabled contract, so anyone configuring limits or reviewing coverage would reasonably assume these routes are protected. Note also that `rateLimiter()` currently reverts on the mainnet hub, the mainnet vault and all four remotes, so nothing is capped anywhere yet — this PR ships the mechanism, and the gap only matters once limits are actually wired.

Recommendation

Include all three deployments in the same rollout. The change is identical to the one already applied and the storage layout allows it with no migration.

If they are deliberately deferred, state in the README which deployments the limiter covers and which it does not, and call out the TAC sUSN address specifically so it is not assumed covered by the sUSN changes.

Status

Resolved

[L-02] StakingVaultOFTUpgradeableHyperlane#redeem - Delegated redemption records the withdrawal claim under the caller instead of the owner

Description

redeem() allows a caller holding an ERC-20 allowance to redeem on behalf of another holder, which is standard ERC-4626 behaviour. The withdrawal queue, however, registers the resulting claim against msg.sender rather than against the owner whose shares were burned. A delegated redemption therefore burns one party's shares and files the claim for another, leaving the owner with nothing to collect. withdraw() rejects delegated calls outright, so the two functions disagree on whether delegation is supported at all.

Vulnerability Details

withdraw() checks if (owner != msg.sender) revert Unauthorized();. redeem() has no equivalent check on either branch, and createWithdrawalDemand files the request under itself:

```
function createWithdrawalDemand(uint256 assets) internal {  
  
    StakingVaultStorage storage s = getStakingVaultStorage();  
  
    if (s.blacklist[msg.sender]) revert BlacklistedAddress();  
  
    IWithdrawalHandler(s.withdrawalHandler).createWithdrawalRequest(msg.sender, assets);  
  
    emit WithdrawalDemandCreated(msg.sender, assets, block.timestamp);  
  
}
```

WithdrawalHandler.claimWithdrawal looks the request up under msg.sender and pays msg.sender, so only the caller can collect. Confirmed by test: when a spender redeems 1,000 shares owned by another holder, no request is created for the owner at all and their claimWithdrawal reverts, while the spender collects the full amount.

If the spender is whitelisted, redeem takes the direct branch instead and the assets transfer immediately, with no queue and no waiting period.

This is not an escalation of what an allowance already permits — a spender could reach the same end state by calling transferFrom and then redeeming normally. The sibling

vault contracts/periphery/StakingVault.sol carries the owner check on both withdraw and redeem, which suggests the omission here is an oversight.

Impact

Misdirected accounting on a legitimate delegation, not theft. An integrator or router that redeems on a user's behalf burns the user's shares and receives the claim itself, so the user has nothing to collect and depends on the integrator forwarding the proceeds. If the integrator is whitelisted it receives the assets immediately, which is harder to reverse.

No attacker gains anything they could not already get through transferFrom, and the exposure is limited to callers a holder has explicitly approved. No current integration appears to call redeem in delegated form, so the issue is latent — but the vault accepts a call pattern it does not handle correctly.

Recommendation

Decide whether delegated redemption is supported, since the contract currently permits it while handling it incorrectly. Either add `if (owner != msg.sender) revert Unauthorized();` to `redeem()` so it matches `withdraw()`, or pass owner to `createWithdrawalDemand` so the claim is filed against the holder whose shares were burned.

Status

Resolved

[L-03] StakingVaultOFTUpgradeableHyperlane - ERC-4626 max* views do not reflect pause, blacklist or the vault's exit restrictions

Description

The four ERC-4626 limit views are inherited unchanged from OpenZeppelin, so they report capacity that the corresponding calls will refuse. While the vault is paused they still report full capacity in both directions, and for a blacklisted holder they still report that holder's entire balance.

Vulnerability Details

EIP-4626 says of `maxDeposit` that it "MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0", with the same wording for the other three. None is overridden here, so the OpenZeppelin defaults apply:

```
function maxDeposit(address) public view virtual returns (uint256) { return
type(uint256).max; }

function maxRedeem(address owner) public view virtual returns (uint256) { return
balanceOf(owner); }
```

While the vault is paused, `maxDeposit` and `maxMint` report unlimited while `deposit` and `mint` revert `EnforcedPause`, and `maxRedeem` still reports the holder's full share balance while `redeem(shares, withdrawalHandler, self)` also reverts `EnforcedPause`. That is the clearest case: exits are entirely disabled, and all four views should read 0.

For a blacklisted holder, `maxRedeem` reports their entire balance while `redeem(shares, withdrawalHandler, self)` reverts `BlacklistedAddress`.

Separately, the exit path is not expressible through these views under any implementation. A non-whitelisted caller must pass the withdrawal handler as the receiver, and a whitelisted caller must not. `maxWithdraw` and `maxRedeem` take only an owner argument, so the receiver is not available to them. This is a documentation matter rather than a max* defect, since a non-reverting call for the full amount does exist.

Impact

No funds are at risk and no state is affected — these are view functions. The consequence is that a generic ERC-4626 integrator sizing a transaction from `max*` builds one that reverts, wasting gas and breaking any interface that relies on the standard. The failure is immediate and explicit, so it surfaces on first use rather than accumulating. Exposure is limited because the exit path is already non-standard in practice, and only the entry side is currently relied on by integrations.

Recommendation

Override `maxDeposit` and `maxMint` to return 0 when the contract is paused or the receiver is blacklisted, and `maxWithdraw` and `maxRedeem` to return 0 when paused or when the owner is blacklisted.

Separately, document in NatSpec and in the integration notes that `withdraw` and `redeem` constrain the receiver — the withdrawal handler for non-whitelisted callers, any other address for whitelisted ones — so integrators do not treat the exit path as standard ERC-4626.

Status

Acknowledged

[L-04] USNUpgradeableHyperlane#handle - Inbound rate limit does not apply to unregistered Hyperlane origins

Description

Every `handle()` authenticates the sender by comparing it against the registered remote token for that origin, but never checks that a remote token was registered at all. For an unregistered origin the lookup returns zero, so a message claiming `sender = 0` passes. That origin's rate limiter bucket was never configured either, and an unconfigured bucket means unlimited. An inbound message from a domain the protocol doesn't bridge with is therefore both accepted and uncapped.

Vulnerability Details

```
// USNUpgradeableHyperlane.sol:192-193

bytes32 expectedToken = remoteTokens[_origin];

if (_sender != expectedToken) revert InvalidRemoteToken();

// BridgeRateLimiter.sol:127

if (r1.limit == 0) return; // unlimited
```

The same missing check appears in `StakingVaultOFTUpgradeableHyperlane.sol:355`, `remote/StakedUSNOFTHyperlane.sol:192` and `remote/StakedUSNHyperlane.sol:176`, and in `periphery/USNHyperlane.sol:131` and `periphery/USNOFTHyperlane.sol:119` — though those last two have no rate limiter at all, so no inbound cap exists there for any origin (see L-01).

Reaching this path needs a forged message. `Mailbox._buildMessage` stamps `sender = msg.sender.addressToBytes32()`, so no honest dispatch can produce a zero sender, including a permissionless dispatch from another chain. It means defeating the validator set for an origin the destination's ISM will route.

That set is wider than the protocol's own routes. As of Ethereum block 26,041,924 the routing component of the mailbox's default ISM covers 74 origin domains, 71 of them unregistered on USN.. The weakest configuration in that table is 2-of-2, and it covers domain 239 (TAC, a registered route) alongside 25 unregistered ones. So the two keys

needed to forge a TAC message also open 25 uncapped doors — an attacker able to abuse the TAC inbound limit is never constrained by it.

Verified by PoC: with a $1,000e18$ inbound limit on a registered origin, a $1,000,000e18$ message reverts and sender = 0 reverts, but the same message on unregistered domain 777777 with sender = 0 mints 3,000,000 USN. On the vault, one such message released 100,000 shares of escrow against a $10e18$ limit.

Impact

No effect under normal operation and no path for an ordinary user — this only becomes reachable if a Hyperlane validator set is compromised, which is an external event the protocol does not control.

What it costs is the mitigation itself. The limiter exists to bound the damage from exactly that scenario, and in that scenario it doesn't bind: the attacker picks an unregistered origin and mints without limit, or drains the vault's escrow in one message. The caps configured on the real routes give less protection than they appear to.

Recommendation

Reject origins that were never registered, in all six receivers:

```
if (expectedToken == bytes32(0) || _sender != expectedToken) revert InvalidRemoteToken();
```

On the four contracts this PR wires, that restores the intended behaviour — a forged message can only come from a registered route, where a configured bucket caps it. On the two periphery contracts it closes the authentication gap, but no cap exists until they are wired.

Status

Resolved

QA

[Q-01] BridgeRateLimiter#checkAndUpdate - Transport parameter is not validated

Description

The limiter validates the transport value in its two owner functions but not in the enforcement function. A transport outside the supported range resolves to a bucket that always allows the transfer and that the owner has no way to configure. No current call site can reach it, so this is a hardening issue rather than a live one.

Vulnerability Details

setRateLimits and resetInFlight both reject anything above TRANSPORT_HYPERLANE, while checkAndUpdate passes the value straight into the bucket key:

```
// setRateLimits and resetInFlight
if (transport > TRANSPORT_HYPERLANE) revert InvalidTransport();

// checkAndUpdate - no equivalent check

RateLimit storage rl = _limits[_key(msg.sender, transport, remoteId, outbound)];

if (rl.limit == 0) return;
```

Since transport is a uint8 and forms part of the key, values 2 through 255 each address a distinct bucket. Those buckets can never be configured, because the owner path rejects the value and the contract is not upgradeable, so they permanently hold limit == 0 and the early return allows any amount through. The validation is present on the path that fails safely and absent on the path that fails open.

All fourteen call sites across the four token contracts pass the hardcoded constants TRANSPORT_LZ or TRANSPORT_HYPERLANE, and the value is never derived from input, so no transport above 1 is reachable today.

Impact

No impact on the deployed system. No caller can pass an unsupported transport, and a third party calling `checkAndUpdate` directly hits the limit `== 0` early return before any storage write, so they change nothing at all. The concern is forward-looking. If a third transport is added later, its route would enforce no limit while simultaneously being impossible to configure, and nothing would surface that state: `checkAndUpdate` emits no event on any path, so an unenforced bucket looks identical to an enforced one on-chain, and `getRateLimit` reports `type(uint256).max` available for it.

Recommendation

Add the same check to `checkAndUpdate` that the owner functions already perform.

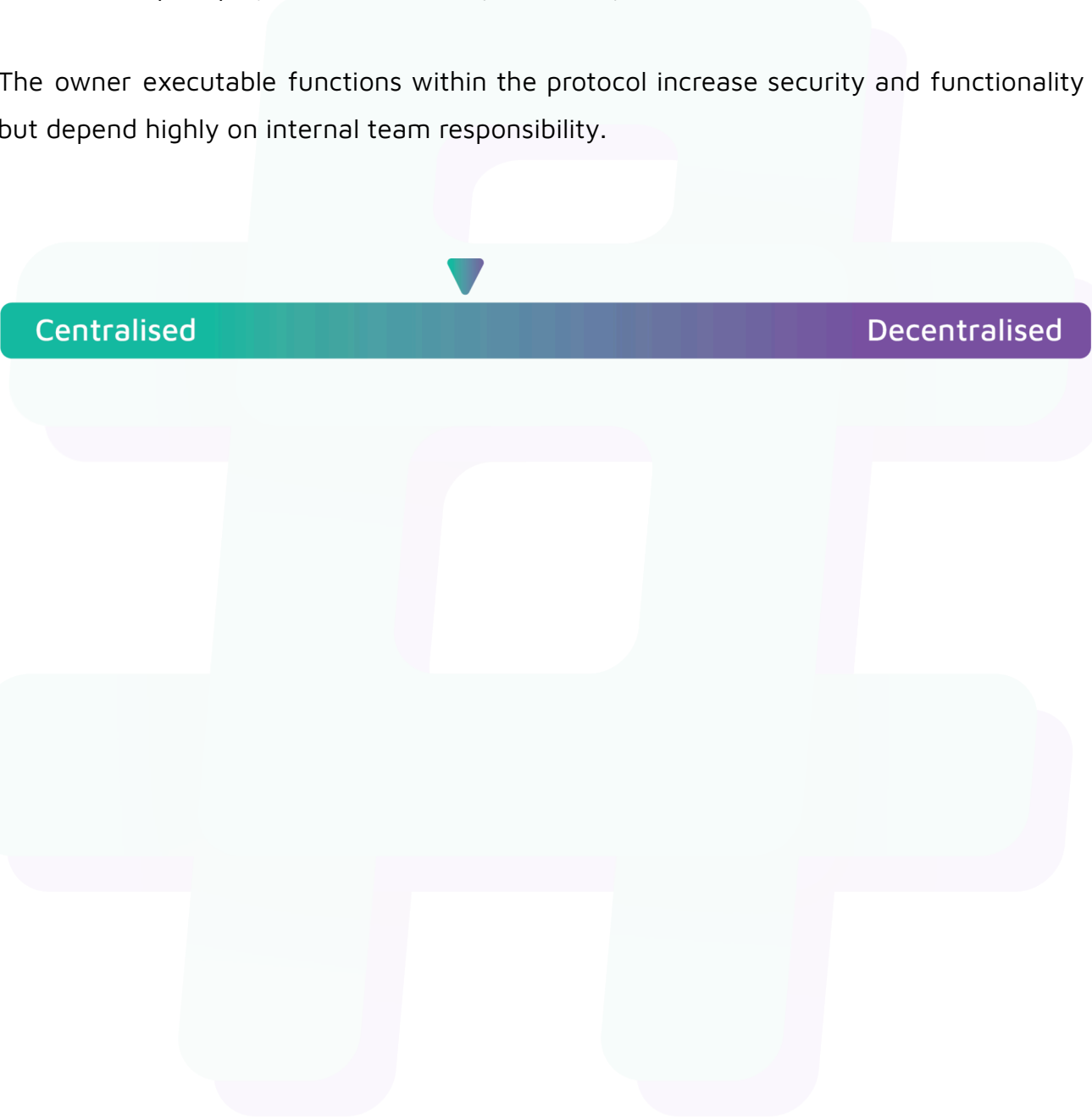
Status

Acknowledged

Centralisation

The Noon Capital project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Noon Capital project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.